

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1104

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:40

Annexure A

Industry Problem Solving Task

Code of Conduct:

I _____S.Poojitha (192320076)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects. Smart Banking and Financial Management System

Ans) Smart Banking and Financial Management System

1. Object-Oriented Principles

- **Encapsulation:** Protects sensitive customer, account, and transaction data.
- **Abstraction:** Hides complex banking operations behind simple interfaces.
- **Inheritance:** Allows common features to be reused among different account types.
- **Polymorphism:** Supports different account, loan, and payment behaviors.
- **Modularity:** Makes the system easier to develop, test, and maintain.

2. Layered Architecture

The system has four layers:

Presentation → Service → Domain → Data Access → OODBMS

- **Presentation Layer:** Handles user interaction.
- **Service Layer:** Performs banking operations such as transfers and loan processing.
- **Domain Layer:** Contains objects such as Customer, Account, Loan, and Transaction.
- **Data Access Layer:** Uses repositories to save and retrieve objects from the OODBMS.

This separation improves **scalability, security, and maintainability** because changes in one layer have minimal impact on others.

3. Design Patterns

- **Factory Pattern:** Creates different account objects such as Savings and Current accounts.
- **Strategy Pattern:** Allows different loan-interest and payment calculation methods.
- **Observer Pattern:** Sends transaction and fraud alerts automatically.
- **Repository Pattern:** Separates database operations from business logic.

4. OODBMS Communication

When a customer performs a transaction, the request moves from **the** Presentation Layer to the Service Layer. The Service Layer updates domain objects such as Account and Transaction. The Repository then stores these objects in the OODBMS. When data is needed, the repository retrieves the persistent objects and sends them back through the domain and service layers to the presentation layer.

Conclusion

Object-oriented principles provide security and reusability, layered architecture provides separation and maintainability, and design patterns provide flexibility and scalability. Together with the OODBMS, they allow the banking system to easily support future services such as digital wallets and investment management.

2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse

stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

Ans) Smart Retail and Inventory Management System

1. Object-Oriented Design Principles

- **Encapsulation:** Protects product, customer, supplier, and stock data.
- **Abstraction:** Hides complex inventory and sales operations.
- **Inheritance:** Allows common features to be reused for different product categories.
- **Polymorphism:** Supports different product and transaction behaviors.
- **Modularity:** Makes the system easier to modify and maintain.

2. Layered Architecture

The system follows:

Presentation → Business → Domain → Data Access → OODBMS

- **Presentation Layer:** Provides interfaces for staff and managers.
- **Business Layer:** Handles sales, purchase orders, and stock operations.
- **Domain Layer:** Contains objects such as Product, Supplier, Customer, and Inventory.
- **Data Access Layer:** Stores and retrieves objects through DAO.

This separation improves **flexibility, scalability, and maintainability**.

3. Design Patterns

- **MVC Pattern:** Separates the user interface, application logic, and data.
- **Factory Pattern:** Creates different product objects without changing existing code.
- **Observer Pattern:** Automatically alerts managers when stock reaches the minimum level.
- **DAO Pattern:** Separates database operations from business logic.

4. OODBMS Interaction

When stock is updated, the Presentation Layer sends the request to the Business Layer. The Business Layer updates the relevant Domain Objects, such as Product and Inventory. The DAO Layer then saves these objects in the OODBMS.

For retrieval, the DAO fetches persistent inventory objects from the OODBMS and passes them through the domain and business layers to the presentation layer. Since the OODBMS stores objects and their relationships directly, inventory data can be retrieved efficiently.

Conclusion

Object-oriented principles provide reusability and flexibility, while layered architecture improves maintainability and scalability. MVC, Factory, Observer, and DAO patterns further simplify development and allow future features such as stock prediction and robotic warehouse management to be integrated easily.

3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

Ans)Smart Hotel and Hospitality Management System

1. Object-Oriented Principles

- **Encapsulation:** Protects guest, reservation, payment, and room information.

- **Abstraction:** Hides complex reservation and payment operations.
- **Inheritance:** Allows common features to be reused for different room and service types.
- **Polymorphism:** Supports different pricing, room, and service behaviors.
- **Modularity:** Makes the system easier to modify and maintain.

2. Design Patterns

- **Strategy Pattern:** Supports different pricing and discount policies without changing the main reservation logic.
- **Factory Pattern:** Creates different room and service objects based on requirements.
- **Observer Pattern:** Sends reservation and room-service notifications to guests or staff.
- **Repository Pattern:** Provides a simple interface for storing and retrieving hotel objects from the OODBMS.

3. Layered Architecture

The system follows:

Presentation → Service → Domain → Data Access → OODBMS

- **Presentation Layer:** Handles guest and staff interactions.
- **Service Layer:** Processes reservations, check-in/check-out, payments, and services.
- **Domain Layer:** Contains objects such as Guest, Room, Reservation, Payment, and Service.
- **Data Access Layer:** Uses repositories to store and retrieve persistent objects.

4. Reservation and OODBMS Processing

When a guest makes a reservation, the Presentation Layer sends the request to the Service Layer. The Service Layer checks room availability and applies the required pricing strategy. The Domain Layer creates or updates the Reservation and Room objects. The Repository then stores these objects in the OODBMS.

When information is needed, the Repository retrieves the required objects from the OODBMS and passes them back through the domain and service layers to the presentation layer.

Conclusion

Object-oriented principles provide reusability, flexibility, and security, while design patterns reduce code duplication and make future changes easier. The layered architecture separates responsibilities and enables efficient communication with the OODBMS. This allows the hotel system to support future features such as smart-room automation and personalized guest services.

4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

Ans) Smart Logistics and Delivery Management System

1. Object-Oriented Principles

- **Encapsulation:** Protects shipment, vehicle, and customer information.
- **Abstraction:** Hides complex delivery and tracking operations.
- **Inheritance:** Allows common features to be reused for different shipment and vehicle types.
- **Polymorphism:** Supports different shipment and delivery behaviors.
- **Modularity:** Makes the system easier to update and maintain.

2. Design Patterns

- **Strategy Pattern:** Selects different delivery routes based on distance, traffic, or delivery time.
- **Factory Pattern:** Creates different shipment objects such as standard, express, or fragile shipments.
- **Observer Pattern:** Sends real-time delivery status notifications to customers and staff.
- **DAO Pattern:** Handles storing and retrieving shipment and vehicle data from the OODBMS.

3. Layered Architecture

The system follows:

Presentation → Business → Domain → Data Access → OODBMS

- **Presentation Layer:** Provides interfaces for customers and logistics staff.
- **Business Layer:** Handles shipment registration, vehicle allocation, routing, and tracking.
- **Domain Layer:** Contains objects such as Shipment, Vehicle, Delivery, and Customer.
- **Data Access Layer:** Uses DAO to communicate with the OODBMS.

4. OODBMS Communication

When a shipment is registered, the request moves from the Presentation Layer to the Business Layer. The Business Layer creates or updates shipment and vehicle objects in the Domain Layer. The DAO then stores these persistent objects in the OODBMS.

For retrieval, the DAO fetches shipment, vehicle, and delivery objects from the OODBMS and passes them back through the domain and business layers to the presentation layer. This provides efficient and organized data management.

Conclusion

Object-oriented principles provide flexibility and reusability, while layered architecture improves scalability and maintainability. Strategy, Factory, Observer, and DAO patterns make the system easier to extend. These features allow future integration of autonomous delivery vehicles and AI-based route optimization.

5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The

architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

Ans) Smart Energy Management System

1. Object-Oriented Principles

- **Encapsulation:** Protects smart-meter, consumption, and customer data.
- **Abstraction:** Hides complex energy-monitoring and analysis operations.
- **Inheritance:** Allows common features to be reused for different meter types.
- **Polymorphism:** Supports different energy-analysis and optimization methods.
- **Modularity:** Makes the system easier to extend and maintain.

2. Design Patterns

- **Observer Pattern:** Sends alerts when abnormal or high energy consumption is detected.
- **Strategy Pattern:** Supports different energy optimization algorithms.
- **Factory Pattern:** Creates different smart-meter objects based on requirements.
- **Repository Pattern:** Provides a simple way to store and retrieve energy objects from the OODBMS.

3. Layered Architecture

The system follows:

Presentation → Business → Domain → Data Access → OODBMS

- **Presentation Layer:** Displays energy dashboards and reports.
- **Business Layer:** Performs consumption analysis and generates alerts.
- **Domain Layer:** Manages objects such as SmartMeter, EnergyUsage, and EnergyAlert.
- **Data Access Layer:** Uses repositories to communicate with the OODBMS.

4. OODBMS Interaction

When a smart meter sends consumption data, the Presentation/Business Layer processes it and updates the relevant domain objects. The Repository stores these objects in the OODBMS.

When energy information is requested, the Repository retrieves the required persistent objects and passes them through the Domain and Business layers to the Presentation Layer. This allows efficient storage and retrieval of energy-monitoring data.

Conclusion

Object-oriented principles provide reusability and flexibility, while layered architecture improves scalability and maintainability. The Observer, Strategy, Factory, and Repository patterns make the system easier to extend. Therefore, future features such as renewable-energy monitoring and AI-based consumption prediction can be added with minimal change

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

Professional ism	5%	Highly professional, well-documented, and clearly presented work	Good documentatio n and presentation	Basic documentation with some gaps	Poor documentatio n and presentation
---------------------	----	--	---	--	---